



TECHNICAL PROJECT REPORT

AI File Assistant

A cross-platform mobile app and Python service that turns raw CSV and Excel files into answers — and lets an AI propose spreadsheet edits that a human always approves.

Flutter client

FastAPI backend

Google Gemini Flash

40 tests passing

DEVELOPED BY

Muhammad Shoaib Asghar

GITHUB

github.com/Shoaib1515

5,400

LINES OF CODE

15

API ENDPOINTS

6

APP SCREENS

40

BACKEND TESTS

STATUS

Feature-complete MVP

RELEASE

v1.0.0

PLATFORMS

Android • iOS • Desktop

Prepared August 2026

Interfaces shown are rendered from the app's own design tokens

01 — EXECUTIVE SUMMARY

A spreadsheet that answers back

AI File Assistant is a complete, working product in two halves: a Flutter mobile client and a FastAPI service. A user uploads a CSV or Excel file, receives an instant data-health report, asks questions about the contents in ordinary language, and — when something needs fixing — describes the change they want and reviews the AI's proposed edits before a single cell is written.

5,400

LINES OF CODE

15

REST ENDPOINTS

6

APP SCREENS

40

AUTOMATED TESTS

The design principle is simple: the model advises, the code decides. Google's Gemini Flash generates every answer and every edit proposal, but no proposal reaches the file until it has passed server-side schema validation and an explicit human approval gate. The AI produces a diff, never a document. That single constraint is what turns a probabilistic model into a tool people can trust with their own data.

Both halves are finished and integrated. The backend exposes fifteen REST endpoints across seven routers — covering authentication, upload, diagnostics, question answering, editing and full file management — and is covered by forty automated tests including a dedicated security-regression suite. The client implements six screens on a hand-built Material 3 design system, keeps sessions alive across restarts, and puts the assistant one tap away from anywhere in the app.

Delivered scope

- ✓ Account creation and sign-in
- ✓ Streaming upload with hard limits
- ✓ Missing-value diagnostics
- ✓ Grounded natural-language Q&A
- ✓ Human review and approval gate
- ✓ Server-side dataset persistence
- ✓ CSV, XLSX and XLS ingestion
- ✓ Format and safety verification
- ✓ Duplicate-row detection
- ✓ AI-proposed cell-level edits
- ✓ Excel write-back and download
- ✓ Re-analysis without re-upload

How the 5,400 lines break down

Mobile client (Dart) 3,582 lines · 15 files: screens, widgets, models, services
Backend application (Python) 1,262 lines · 7 routers, 4 services, 2 core modules
Automated test suite (Python) 556 lines · 40 test cases across 11 files

02 — THE PROBLEM

Everyone owns data. Few can interrogate it.

Spreadsheets are the most widely used database on earth and the least inspected. A file arrives by email, is edited by four people, and quietly accumulates blank cells, duplicated rows and inconsistent formatting. The damage stays invisible until a total comes out wrong.

The tools that could find those problems are gated behind expertise. Pivot tables, lookup formulas and data-science notebooks all assume the person holding the data is also a technician. Often they are not — they are a shop owner, a coordinator, a teacher, a researcher against a deadline. They know exactly what they want to ask and have no syntax to ask it with.

Generative AI looks like the obvious bridge, and for *reading* a dataset it is. For *changing* one it introduces a new fear: a model that writes directly to a file can silently corrupt hundreds of rows, and a plausible-looking mistake is the hardest kind to catch. A serious AI editing feature therefore has to answer a harder question than “can the model do it” — it has to answer “what happens when the model is wrong.” AI File Assistant is built around that second question.

Five frustrations, five answers

The everyday frustration	How AI File Assistant answers it
“I don't know if this file is even clean.”	One tap returns row and column totals, a per-column count of missing values, and every duplicate-row group — each shown with sample rows so the finding is verifiable, not abstract.
“I have a question but not the formula.”	The assistant answers in plain language, grounded on the dataset's real measured summary rather than guesswork, so the numbers in the reply match the numbers in the file.
“The fix is repetitive and error-prone.”	The user describes the change once — for example, adding a country code to every phone number — and the model returns the exact list of cells it would touch.
“I can't let an AI loose on my data.”	Every proposal is validated against the real column set and shown as a before / after diff. Approve and it applies; cancel and the stored file is left byte-for-byte unchanged.
“My work vanishes when I close the app.”	Files and their metadata are persisted server-side under the owning account, so a dataset can be re-analysed or edited days later without being uploaded again.

03 — CAPABILITIES

What the application actually does

Six capabilities make up the product. Each is fully implemented end to end — from a Flutter screen, through a REST endpoint, to a Python service and back — and each maps to a concrete moment in the user's workflow.

01 Universal file ingestion

Bring almost any tabular file; the server does the rest.

- Accepts CSV, XLSX and XLS via a native file picker
- Streamed in 64 KB chunks with a hard 10 MB ceiling

02 One-tap data diagnostics

Instant health check the moment a file lands.

- Row / column totals and per-column missing-value counts
- Exact duplicate-row detection with sample offending rows

03 Grounded natural-language Q&A

Ask about the data in plain words; get grounded answers.

- Questions answered against the real measured dataset summary
- Prompt hardened so file content is treated as data, not instructions

04 AI-assisted safe editing

Describe a change; approve a diff before anything is written.

- Model returns a structured list of cell changes, not a file
- Nothing is applied until the user approves the before / after view

05 Persistence and history

Datasets outlive the session and stay one tap away.

- Files and metadata stored server-side under the owner
- Re-analyse or edit later with no re-upload

06 Accounts and isolation

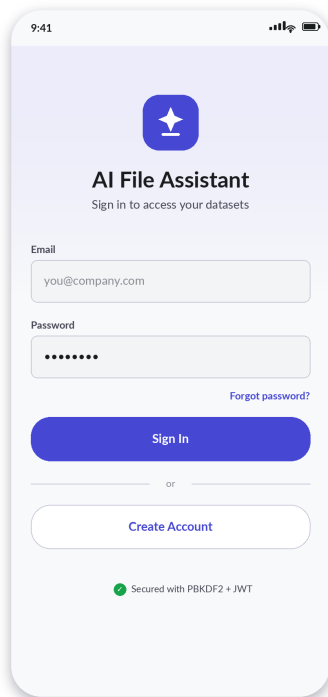
Every file belongs to exactly one account.

- Email + password sign-up with hashed credentials and a signed token
- Every dataset query is scoped to the authenticated owner

04 — THE INTERFACE

Six screens, one design language

The client is built on a single hand-authored Material 3 design system: one indigo brand colour, a consistent type scale, and a shared component vocabulary of cards, chips and bottom navigation. The screens below are rendered directly from the app's own design tokens.

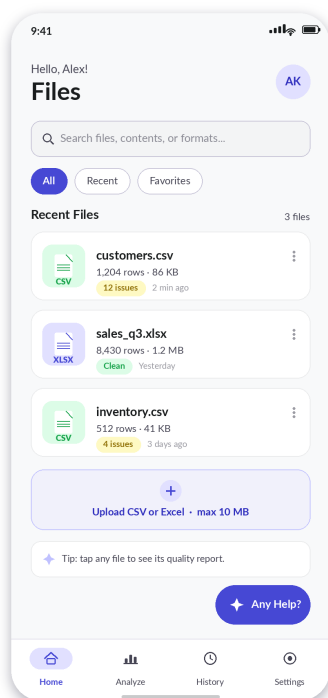


01 Sign in

Secure entry point

Email-and-password authentication backed by a hashed credential store and a signed 7-day session token.

The token is kept on the device, so a returning user lands straight on their files without signing in again.

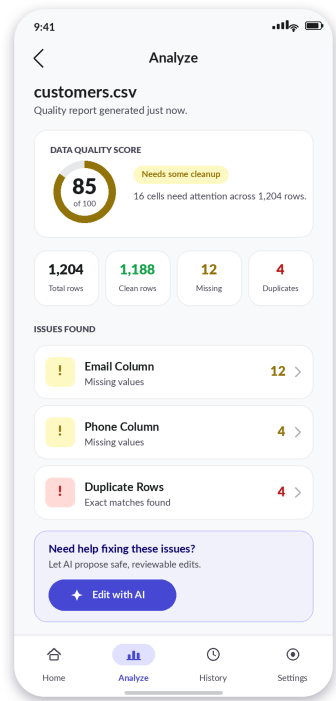


02 Home & file library

Every dataset in one place

The landing screen lists previously uploaded files with row and column counts, and offers a single upload button.

Tapping a file reopens it for analysis or editing — no re-upload, because the server kept it.

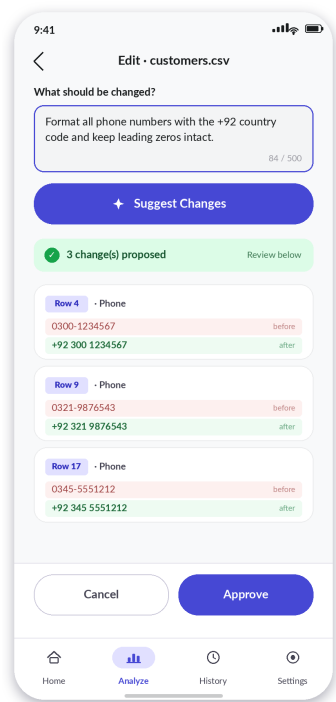


03 Analyze

The data-health report

A quality score, headline tiles for rows, columns and missing cells, and an issue list for every problem found.

Duplicate groups and empty columns are shown with sample rows, so the diagnosis can be checked against reality.

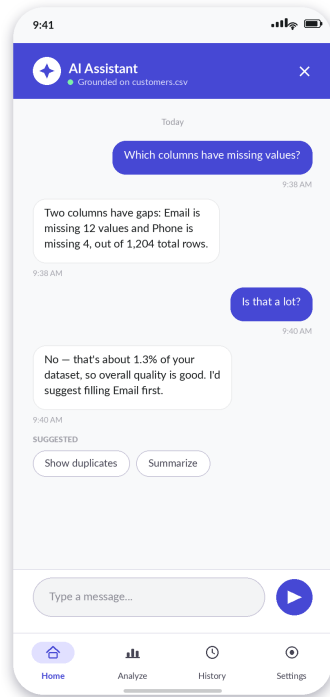


04 Edit & review

Where the approval gate lives

The user types an instruction; the AI returns proposed cell changes rendered as a before / after list.

Apply writes the edits back to a new .xlsx; cancel leaves the stored file untouched. The human is always the switch.



05 Assistant

Conversation, one tap away

A floating assistant reachable from anywhere in the app, for asking questions about the active dataset in plain text.

Answers are grounded on the file's measured summary, so the figures in a reply line up with the figures in the data.

Two more screens complete the set

History keeps a running list of every dataset opened, so past work is one tap away. Settings holds the account details and sign-out. All six screens draw from the same Material 3 tokens, so colour, type and spacing stay consistent throughout the app.

05 — ARCHITECTURE

How the system is put together

The system is a clean three-tier design. A Flutter client talks to a FastAPI service over HTTPS with a bearer token; the service owns all business logic and delegates only to three external providers — Google Gemini for language, a relational database for metadata, and the local filesystem for the datasets themselves.

A single gateway on each side

On the client, every network call funnels through one `ApiService` class, so authentication headers, base URLs and error handling live in exactly one place. On the server, FastAPI routers are thin: they validate the request, call a service, and shape the response. All the real work happens in the service and core layers, which keeps the code testable and the responsibilities clear.

That separation is deliberate. Because parsing, analysis, AI prompting and editing each live in their own service module, any one of them can be tested in isolation and swapped without disturbing the HTTP layer above it — the reason the backend can carry forty tests without brittle setup.

The module map below lists every backend building block and what it is responsible for.

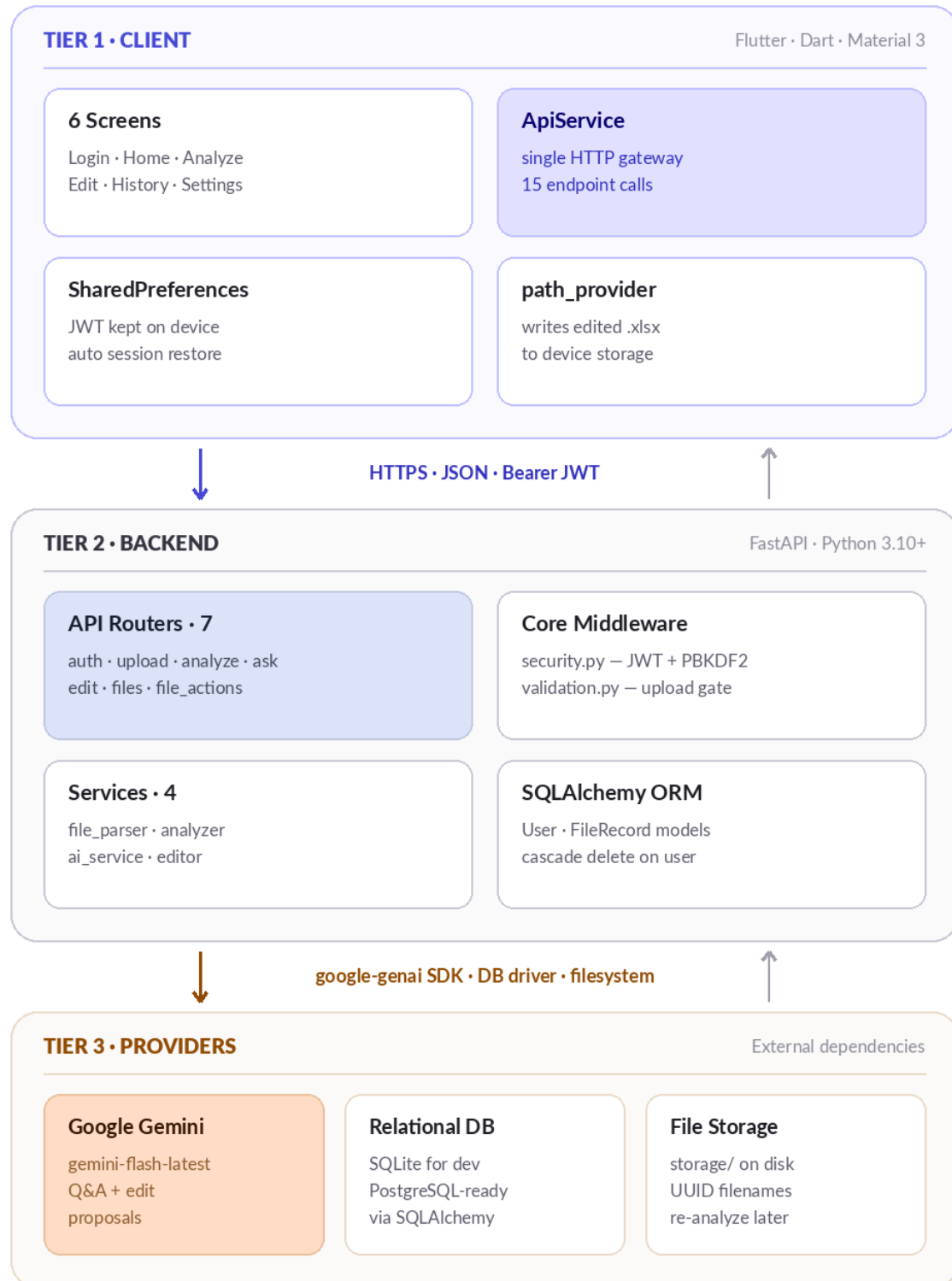
Layer	Module	Responsibility	LOC
Entry	main.py	App startup, router wiring, health check	39
API	api/ (7 routers)	Auth, upload, analyze, ask, edit, files, file-actions endpoints	555
Core	core/security.py	Password hashing, token signing, auth dependency	118
Core	core/validation.py	Upload gate, formula sanitising, AI-output checks	141
Service	services/ai_service.py	Gemini prompting, grounding, retries	188
Service	services/analyzer.py	Missing values and duplicate detection	79
Service	services/editor.py	Applies approved changes, writes .xlsx	73
Service	services/file_parser.py	Parses CSV / Excel into a DataFrame	54
Data	models/ + db/	SQLAlchemy models and the session factory	71

From modules to runtime

The diagram on the next page shows how these building blocks connect when a request runs — which tier calls which, over what protocol, and where the trust boundaries sit between them.

System Architecture

Three tiers: Flutter client, FastAPI service layer, external providers



Every dataset route is guarded by `get_current_user` and scoped to the authenticated owner, so one account can never read or edit another account's files.

Figure 1 — The three tiers, their internal modules, and the boundaries between them.

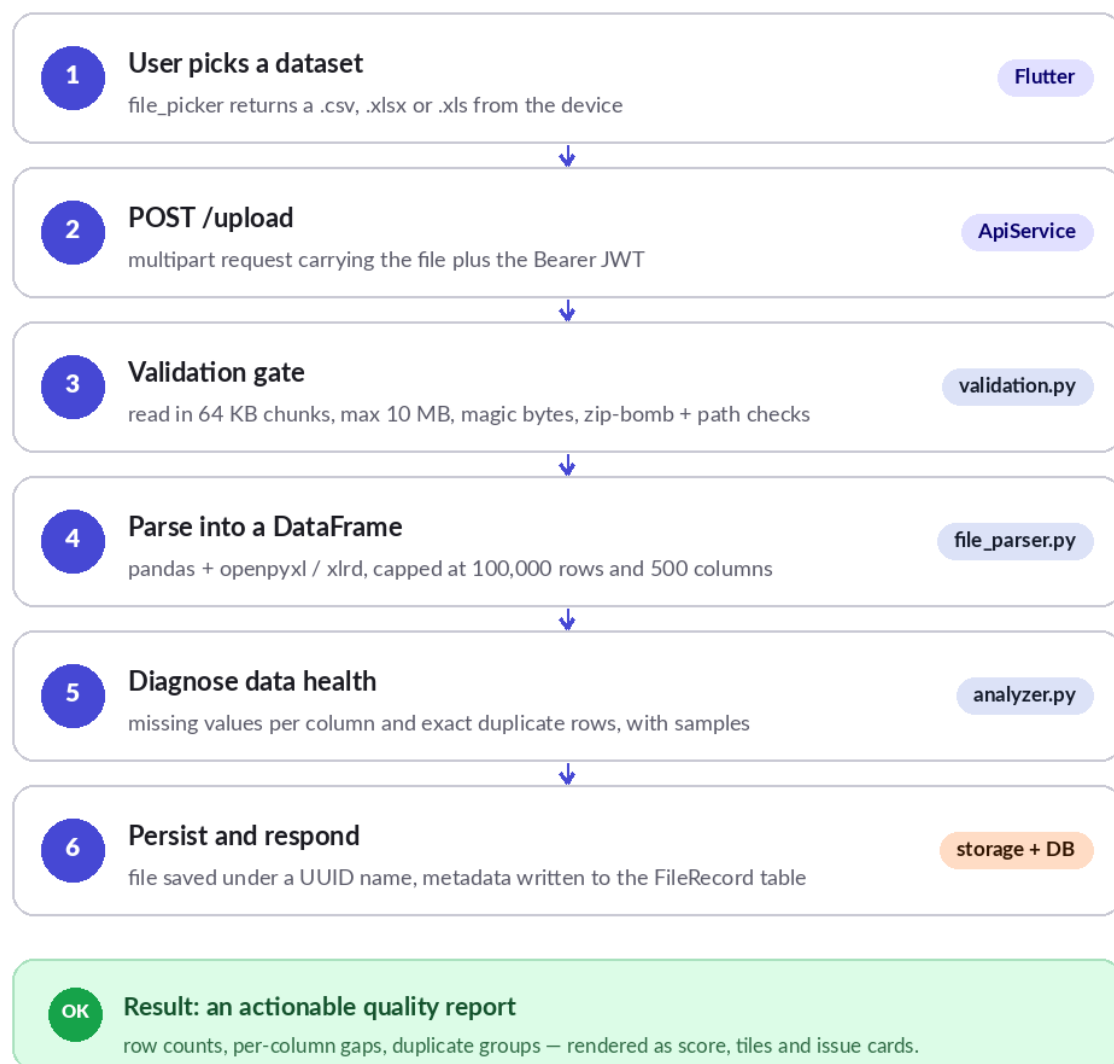
06 — DATA FLOWS

Following the data, end to end

Three flows carry the whole product: getting a file in and diagnosed, answering a question about it, and editing it safely. Each is traced below from the tap that starts it to the result that comes back — with the responsible module named at every step.

Data Flow 1 — Upload and Diagnose

What happens between choosing a file and seeing its quality report



The dataset is never trusted on arrival: size, format and internal structure are all verified before pandas is allowed to read it.

Figure 2 — Upload and diagnose: the file is verified before pandas is ever allowed to read it.

When a question is asked, the dataset's stored summary travels with it so the model answers about the real file rather than inventing plausible numbers. The dataset content is wrapped in explicit delimiters and marked as untrusted input, which is what stops a malicious spreadsheet from hijacking the prompt.

Data Flow 2 – Grounded AI Q&A

How a plain-language question becomes an answer about the real data

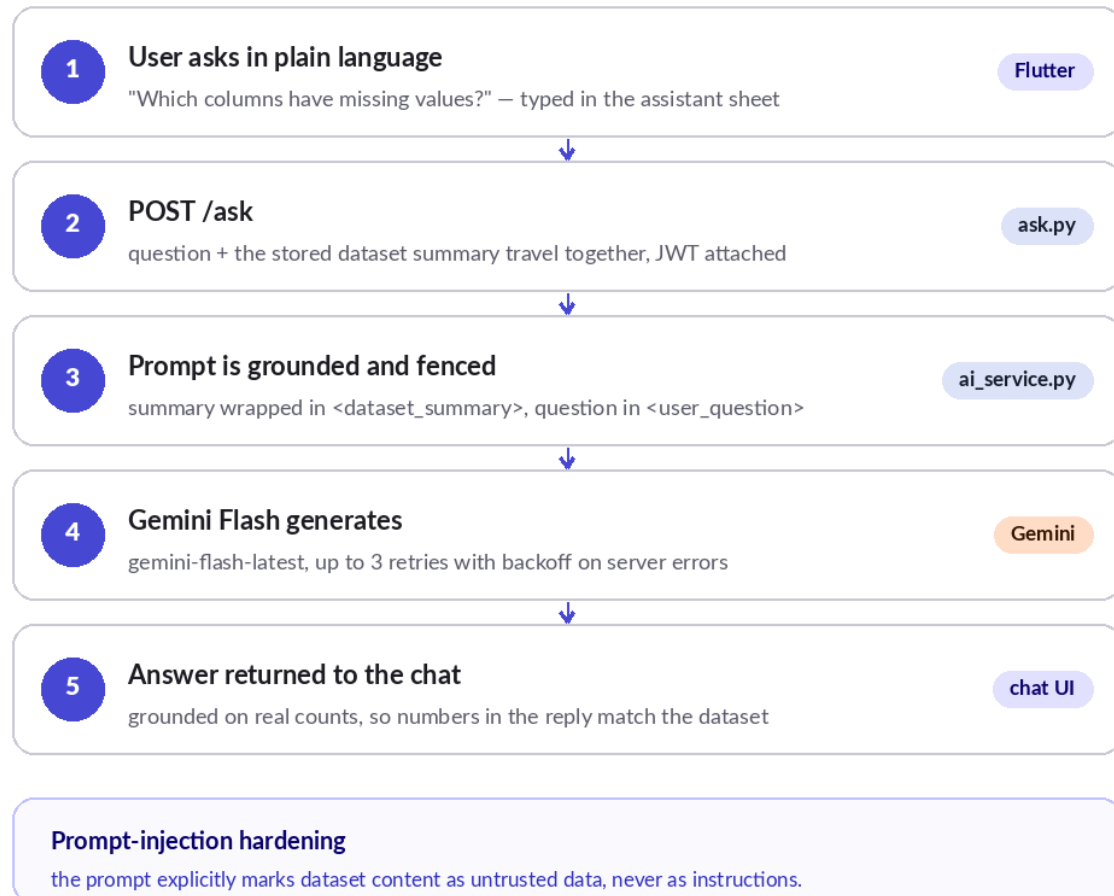


Figure 3 – Grounded Q&A: real measured context in, a grounded answer out.

Editing is where the architecture earns its keep. The model never receives write access; it returns a proposal, the server validates that proposal against the real schema, and the user sees a diff before anything happens. The two outcomes — apply or cancel — are the only ways forward, and cancelling is completely inert.

Data Flow 3 — AI-Assisted Safe Editing

The AI proposes, the human decides: no cell changes without approval



The model can only ever produce a proposal — an incorrect suggestion can never silently corrupt user data.

Figure 4 — Safe editing: the AI proposes, validation screens, and the human decides.

07 — API SURFACE

Fifteen endpoints, seven routers

The backend's contract is small and deliberate. Every dataset route requires a valid bearer token and is scoped to the authenticated owner, so the same endpoint can never reach another account's files.

Method	Path	What it does	Auth
GET	/	Service health check	–
POST	/auth/register	Create an account	–
POST	/auth/login	Sign in, receive a token	–
GET	/auth/me	Current user profile	Token
POST	/upload	Upload, validate, diagnose, store a file	Token
POST	/analyze	Diagnose an in-memory dataset	Token
POST	/ask	Grounded natural-language question	Token
POST	/suggest-edit	Request AI cell-change proposals	Token
POST	/apply-edit	Apply approved changes	Token
GET	/files	List the user's stored files	Token
GET	/files/{id}/download	Download a stored file	Token
DELETE	/files/{id}	Delete a stored file	Token
POST	/files/{id}/analyze	Re-analyse a stored file	Token
POST	/files/{id}/suggest-edit	Propose edits on a stored file	Token
POST	/files/{id}/apply-edit	Apply edits to a stored file	Token

08 — DATA MODEL

What is stored, and how it stays private

Persistence is intentionally minimal: two tables, joined by ownership. The dataset files live on disk under generated names; the database stores only the metadata needed to find, list and re-open them.

users

Column	Type
id	int, PK
email	unique, indexed
hashed_password	string
full_name	string
created_at	datetime

files

Column	Type
id	int, PK
user_id	FK → users
filename	string
storage_path	string
total_rows / columns	int
missing_values	JSON
uploaded_at	datetime

Ownership is enforced, not assumed

The relationship between the two tables is declared with a cascade delete: removing a user removes their files with them, so no orphaned records are left behind. More importantly, every query that reads or mutates a file filters by the current user's id. A request for someone else's file id simply returns nothing — the classic insecure-direct-object-reference hole is closed at the query level rather than patched at the edges.

Because the schema talks to SQLAlchemy rather than a specific engine, the same code runs on SQLite during development and on a production database without changes to the application layer.

09 — SECURITY

Built to distrust its own inputs

Security here is not a bolt-on; it is the reason several of the modules exist. The system treats uploaded files, model output and client requests as untrusted by default, and each gets a specific, tested defence.

Concern	How it is handled
Password storage	PBKDF2-HMAC-SHA256 with a random per-user salt and 100,000 iterations; raw passwords are never stored.
Session tokens	Signed HMAC-SHA256 tokens verified in constant time, with a fixed expiry, so a tampered token is rejected.
Malicious uploads	File type confirmed by magic bytes, not extension; size capped by streaming; zip-bomb and path-traversal guards on archives.
Formula injection	Cells beginning with = + - @ are prefixed on export so spreadsheets cannot execute them on open.
Prompt injection	Dataset content is delimited and explicitly marked as untrusted data, never interpreted as instructions to the model.
Untrusted AI output	Every proposed change is schema-checked against real columns and bounded in count before it can be applied.
Cross-account access	All dataset queries are scoped to the authenticated owner, closing insecure direct object references.

10 — TESTING & QA

Forty tests, including the attacks

The backend ships with a real automated test suite — not a smoke test. Forty test cases across eleven files exercise the services, the API layer, the error paths and, notably, a dedicated security-regression file that re-runs known attack shapes to make sure old holes stay closed.

40

TEST CASES

11

TEST FILES

556

LINES OF TESTS

7

AREAS COVERED

Test file	Cases
test_ai_service.py	10
test_ai_integration.py	4
test_analyzer.py	4
test_editor.py	4
test_upload.py	4
test_files.py	3

Test file	Cases
test_analysis_api.py	3
test_auth.py	2
test_edit_api.py	2
test_error_handling.py	2
test_security_regression.py	2

What the tests protect

The suite covers the AI service's prompt construction and retry behaviour, the analyzer's counting logic, the editor's write path, and the full request cycle for upload, analysis and editing. The security-regression file is the one that matters most over time: it encodes the specific abuses the system was hardened against, so a future refactor that re-opens one of them fails loudly instead of shipping quietly.

11 — TECHNOLOGY

The stack, end to end

Every dependency is a current, mainstream choice, pinned to a known version. The client is idiomatic Flutter; the server is idiomatic FastAPI; the two meet over plain JSON.

Mobile client

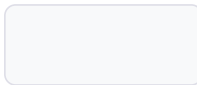
Package	Role
Flutter / Dart	UI framework
http	REST calls
file_picker	File selection
shared_preferences	Token storage
path_provider	File output

Backend service

Package	Role
FastAPI / Uvicorn	Web service
google-genai	Gemini access
pandas	Data handling
openpyxl / xlrd	Excel I/O
SQLAlchemy	ORM / database

Design language

The interface is built on one Material 3 palette, defined once as design tokens and reused across all six screens.

					
Primary #4648D4	Primary variant #2F2EBE	Tertiary #904900	Success #16A34A	Error #BA1A1A	Surface #F8F9FA

In one sentence

AI File Assistant is a finished, tested, cross-platform app that lets anyone clean and question a spreadsheet in plain language — with an AI that can only ever propose changes, and a human who always has the final say.